

1. Unrolled List
–Insert Element
–Program Analysis

Program:

```
#define MAX_ELEMENTS 4

// Structure for the Unrolled Linked List Node
struct Node {
    int numElements;
    int elements[MAX_ELEMENTS];
    struct Node* next;
};

// Global head pointer
struct Node* head = NULL;

void insertElement(int val) {
    if (head == NULL) {
        head = (struct Node*)malloc(sizeof(struct Node));
        head->numElements = 1;
        head->elements[0] = val;
        head->next = NULL;
        cout << "Inserted " << val << " into a new head node.\n";
        return;
    }
    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    if (temp->numElements < MAX_ELEMENTS) {
        temp->elements[temp->numElements] = val;
        temp->numElements++;
        cout << "Inserted " << val << " into the existing last
node.\n";
    }
}
```

```

else {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->numElements = 1;
    newNode->elements[0] = val;
    newNode->next = NULL;

    temp->next = newNode;
    cout << "Node full. Inserted " << val << " into a newly
created node.\n";
}
}
...
    case 1:
        cout << "Enter integer value to insert: ";
        cin >> value;
        insertElement(value);
        break;

```

Program Analysis

Head globally declared NULL ,through:

```
struct Node* head = NULL;
```

1. When the program starts:

*Every time we double – click the executable or run it from the terminal,
the operating system gives the program a completely fresh block of memory.*

*The C + + runtime looks at struct Node * head = NULL;*

and sets that memory address to NULL (0).

Because of this, every completely new run of the program starts with a perfectly clean, empty list.

2. *head == NULL means the list is empty:*

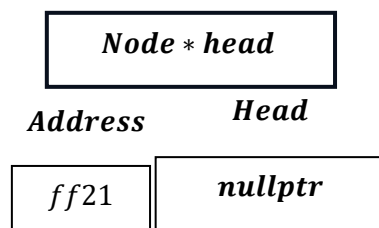
*In linked list design, the **head** pointer represents the doorway to our data.*

*If that doorway leads to nowhere (**NULL**), it is the mathematical guarantee that zero nodes exist in memory. That is why functions like **insertElement** and **deleteElement** always check if (**head == NULL**) first.*

***Head** is only automatically NULL the very first millisecond the program launches.*

After that, it only becomes NULL again if we explicitly make it NULL in our code .

*Therefore , during the start of our program , **Head** is **NULL**.*



```
cout << "Enter integer value to insert: ";
cin >> value;
insertElement(value);
break;
```

``Enter integer value to insert: ``

say, 20

*Function Call: **insertElement**(**value**: 20)*

```

if (head == NULL) {
    head = (struct Node*)malloc(sizeof(struct Node));
    head->numElements = 1;
    head->elements[0] = val;
    head->next = NULL;
    cout << "Inserted " << val << " into a new head node.\n";
    return;
}

```

∴ ***if(Head = NULL) is TRUE :***

head = (struct Node *)malloc(sizeof(struct Node));

And we have:

```

struct Node {
    int numElements;
    int elements[MAX_ELEMENTS];
    struct Node* next;
};

```

On 32 – bit architecture:

- ***int numElements;*** = 4 bytes
- ***int elements[4];*** = 16 bytes (4 integers × 4 bytes)
- ***struct Node * next;*** = 4 bytes (on 32 – bit)

Total memory allocated: 24 bytes

On 64 – bit architecture:

- ***int numElements;*** = 4 bytes
- ***int elements[4];*** = 16 bytes (4 integers × 4 bytes)
- ***Padding :*** 4 bytes to align with 8 byte pointer variable ***next***.
- ***struct Node * next;*** = 8 bytes (on 64 – bit)

Total memory allocated: 32 bytes

Most modern system architecture follows 64 – bit , hence :

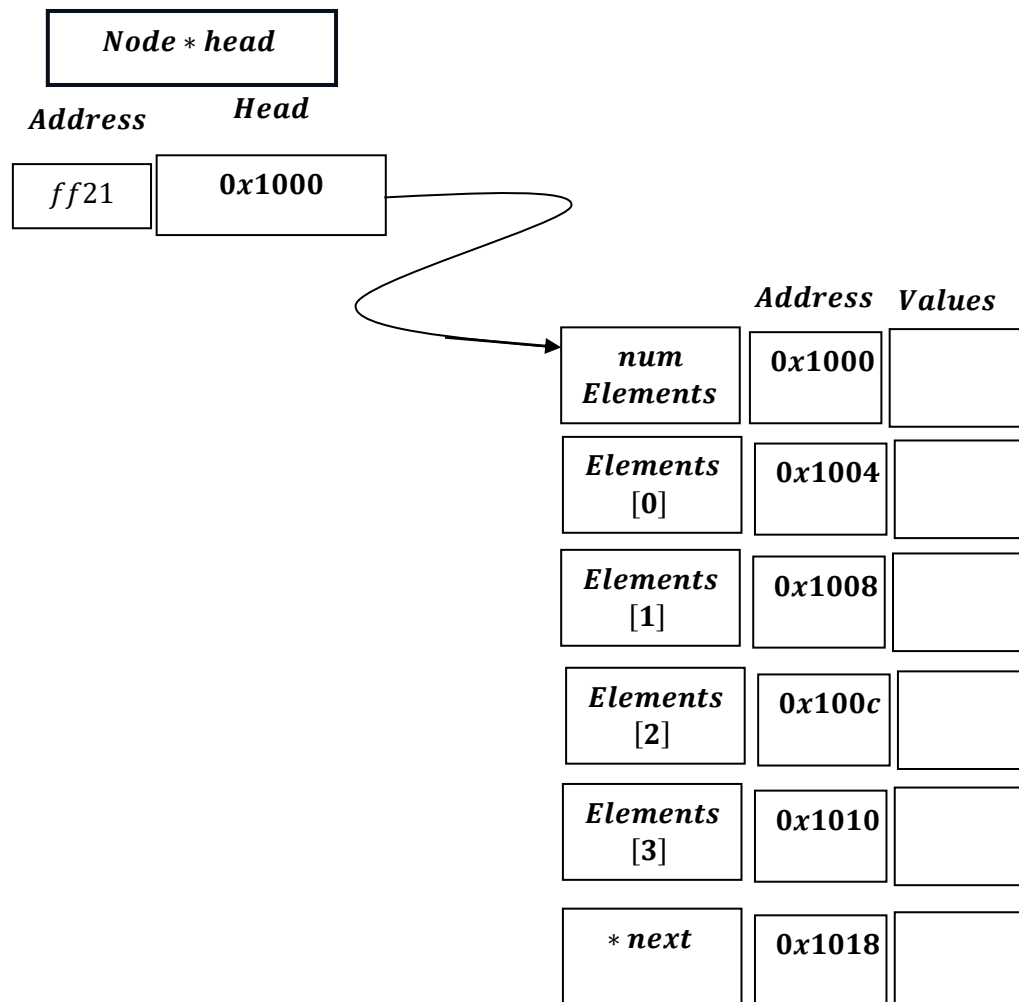
<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
0x1000	<i>E.g.: numElements = 1, 2 ... etc.</i>	<i>numElements[0x1000 – 0x1003] 4 bytes for int</i>
0x1004	<i>E.g.: elements = {15, 42, ... etc. }</i>	<i>elements[0x1004 – 0x1013] 16 bytes for int array</i>
		<i>padding → [0x1014 – 0x1017] 4 bytes for padding</i>
0x1018	<i>E.g.: Address: 0x2000</i>	<i>* next → [0x1018 – 0x101F] 8 bytes for pointer variable</i>

Where as ,we have elements table like:

<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
0x1004	<i>elements[0]</i>	<i>elements[0] → [0x1004 – 0x1007]</i> <i>4 bytes for int</i>
0x1008	<i>elements[1]</i>	<i>elements[1] → [0x1008 – 0x100b]</i> <i>4 bytes for int</i>
0x100c	<i>elements[2]</i>	<i>elements[2] → [0x100c – 0x100f]</i> <i>4 bytes for int</i>
0x1010	<i>elements[3]</i>	<i>elements[3] → [0x1010 – 0x1013]</i> <i>4 bytes for int</i>

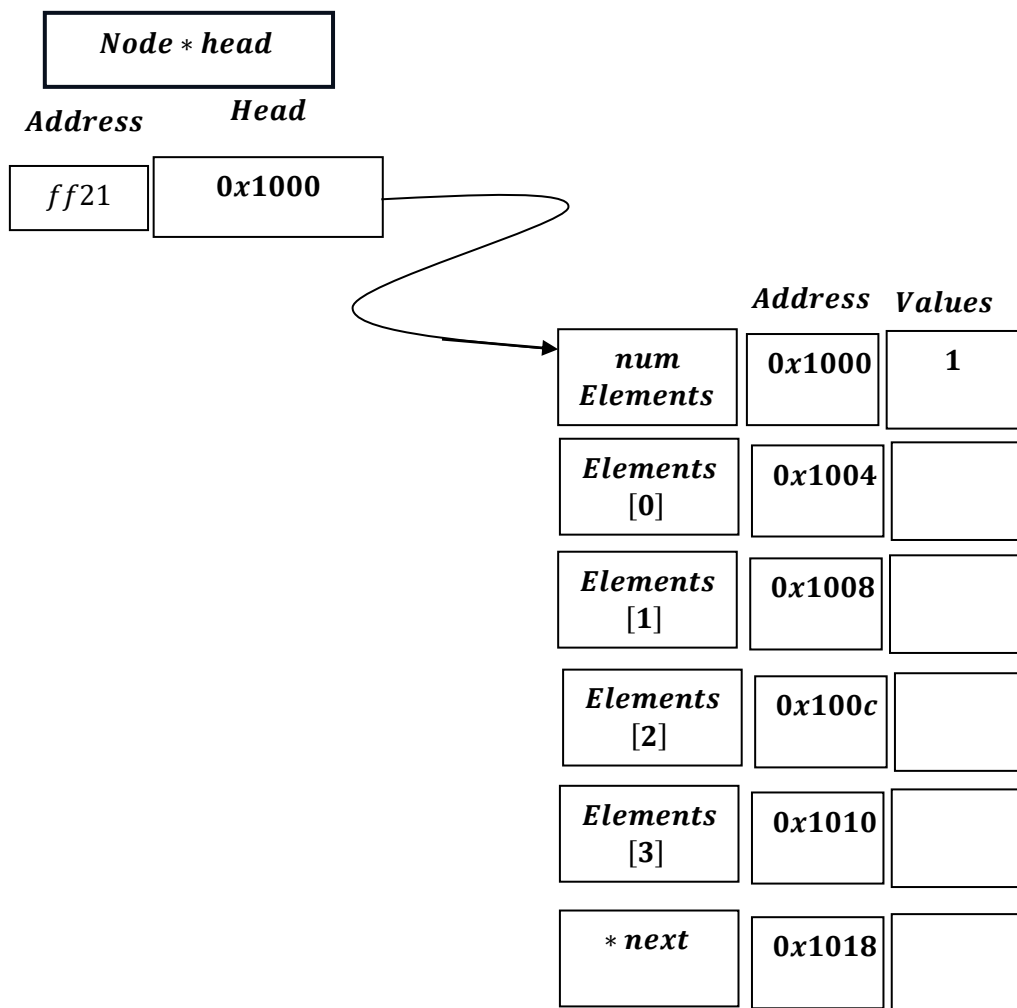
Now, head will start to point to the first element i. e. NumElements

–an integer type of the struct .



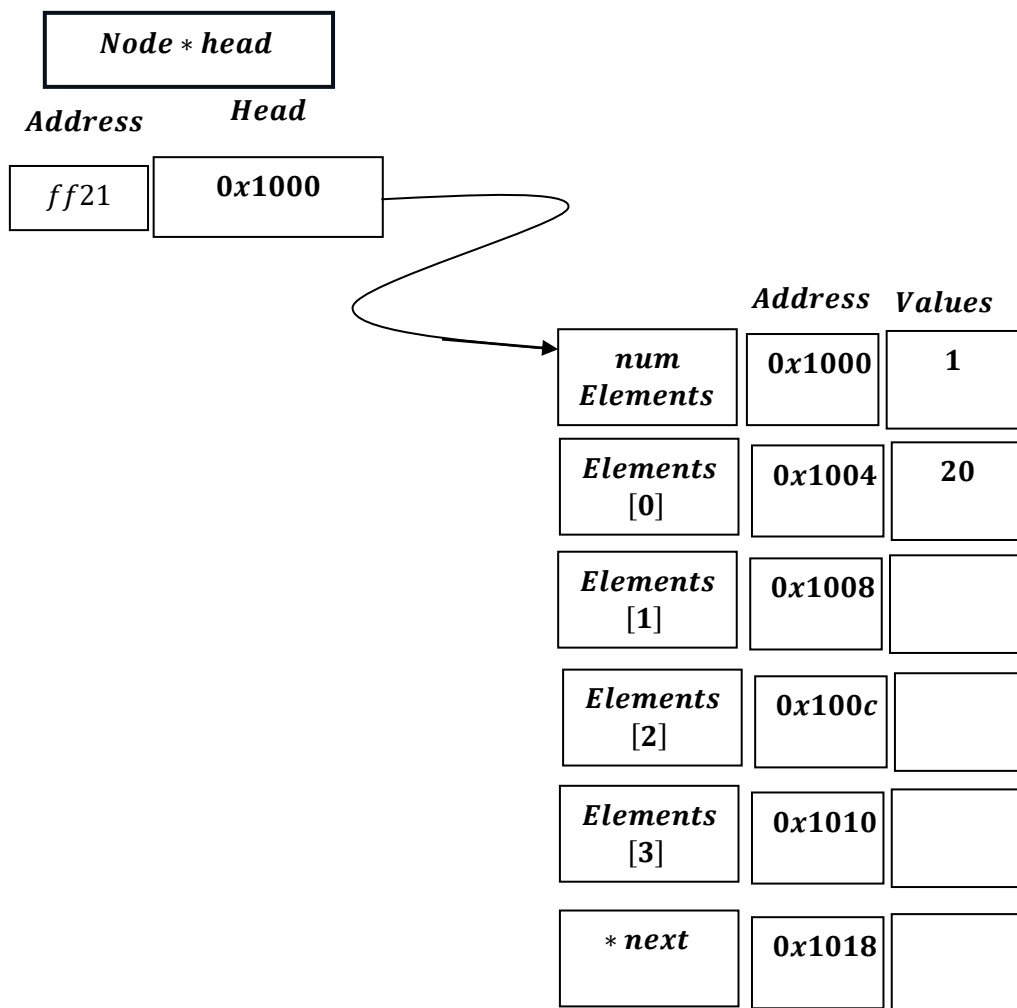
```
head->numElements = 1;
```

We get:



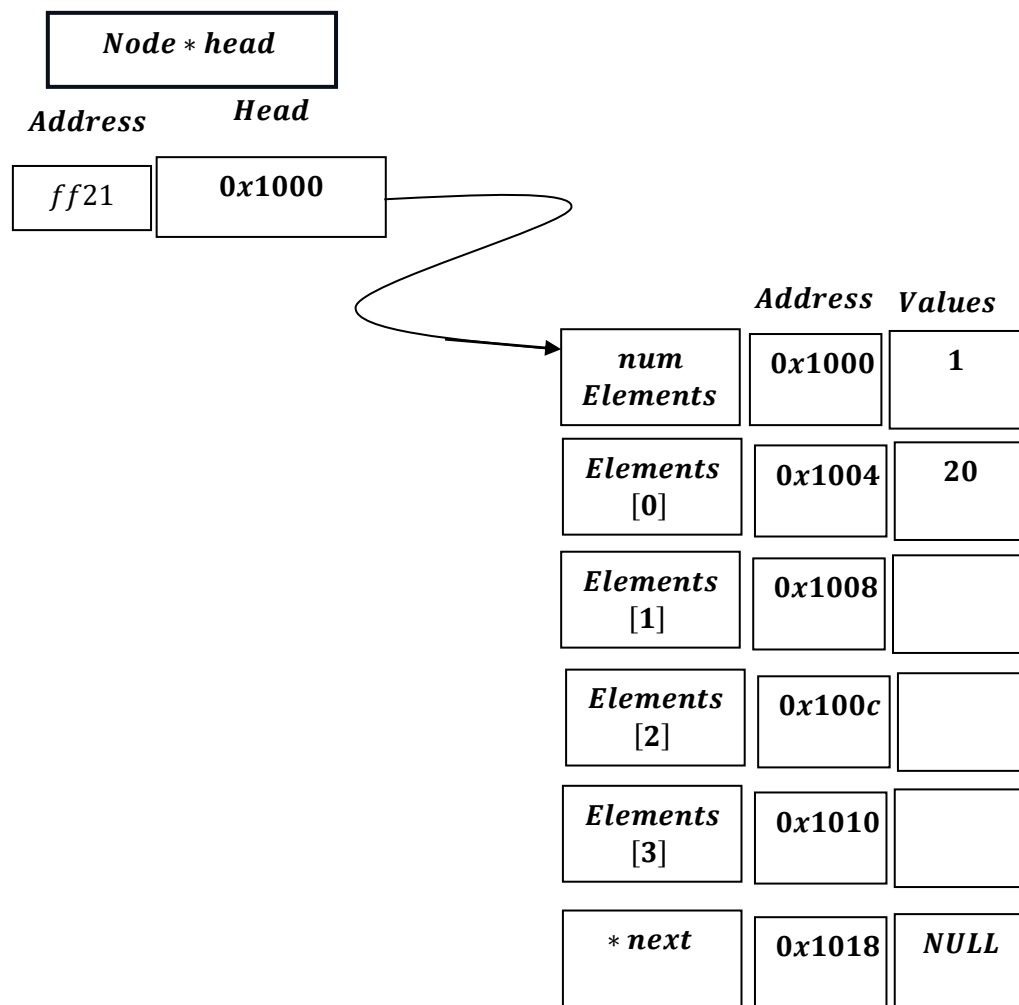
```
head->elements[0] = val;
```

Now, our *val* is : **20** (during function call)



```
head->next = NULL;
```

head → *next* = *NULL*;



Again say,

```
cout << "Enter integer value to insert: ";
cin >> value;
insertElement(value);
break;
```

``Enter integer value to insert: ``

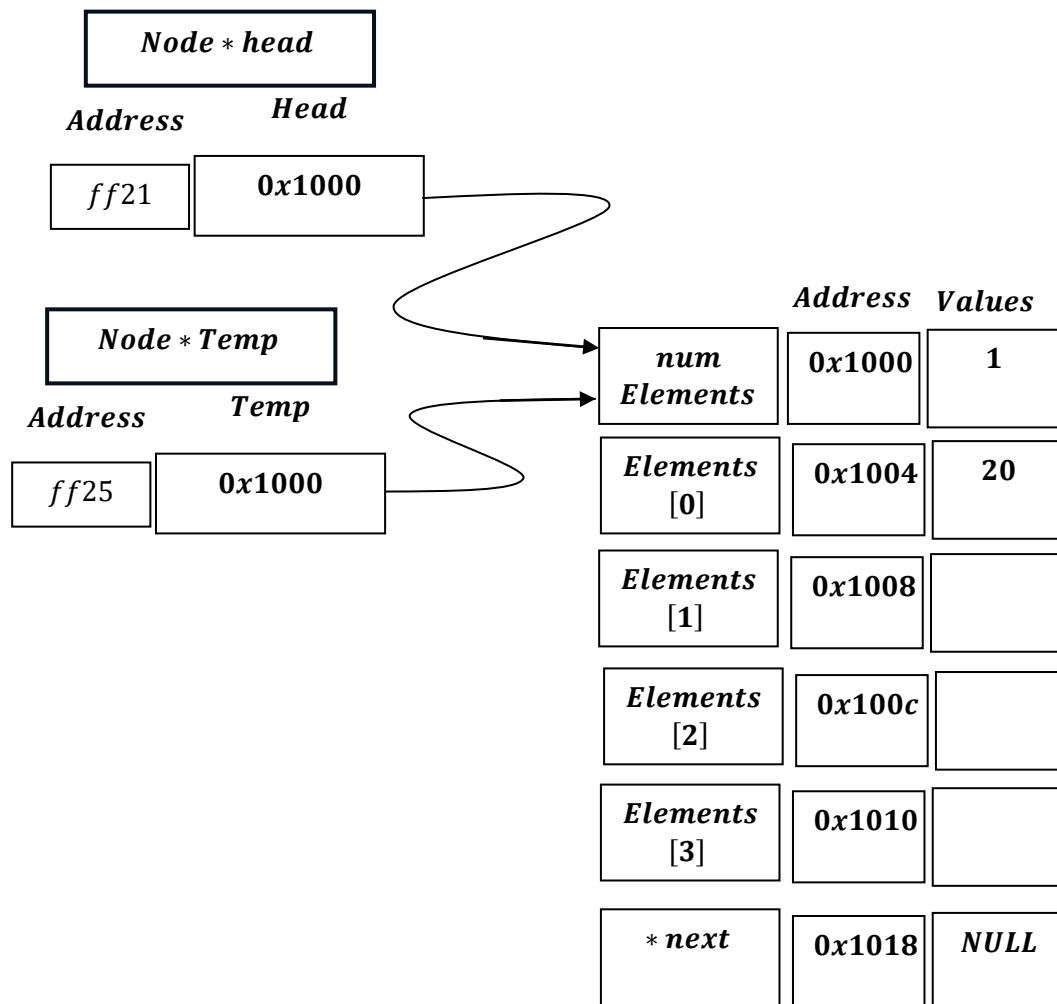
say, 30

Function Call: insertElement(**value**: 30)

Now ,Head is not NULL hence :

```
struct Node* temp = head;
```

*struct Node * temp = head = 0x1000;*



```
while (temp->next != NULL) {  
    temp = temp->next;  
}
```

TEMP → NEXT = NULL ,hence condition of the while loop is false .

Therefore ,loop exists and goes to next line of code.

```

if (temp->numElements < MAX_ELEMENTS) {
    temp->elements[temp->numElements] = val;
    temp->numElements++;
    cout << "Inserted " << val << " into the existing last
node.\n";
}

```

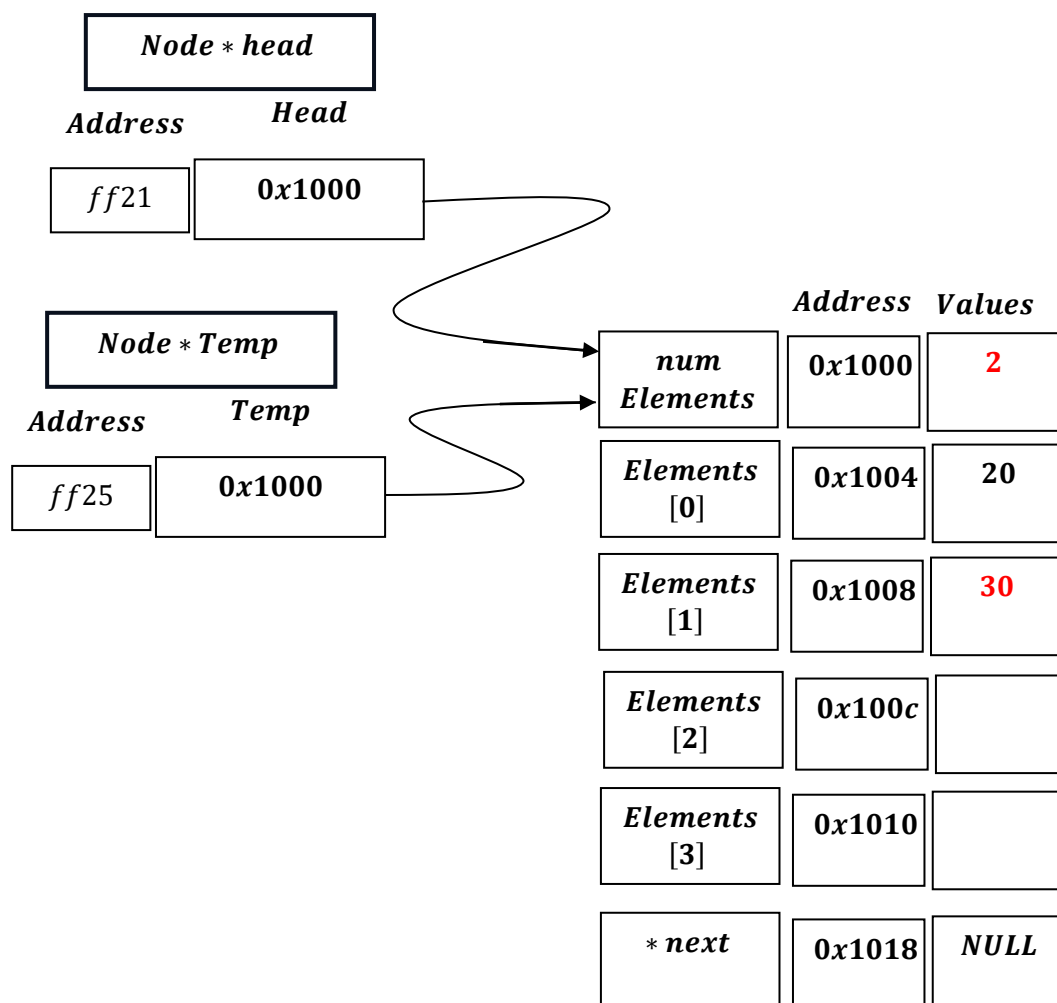
$if (temp \rightarrow numElements = 1 < MAX_ELEMENTS = 4) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$temp \rightarrow elements[temp \rightarrow numElements = 1] = val := 30;$

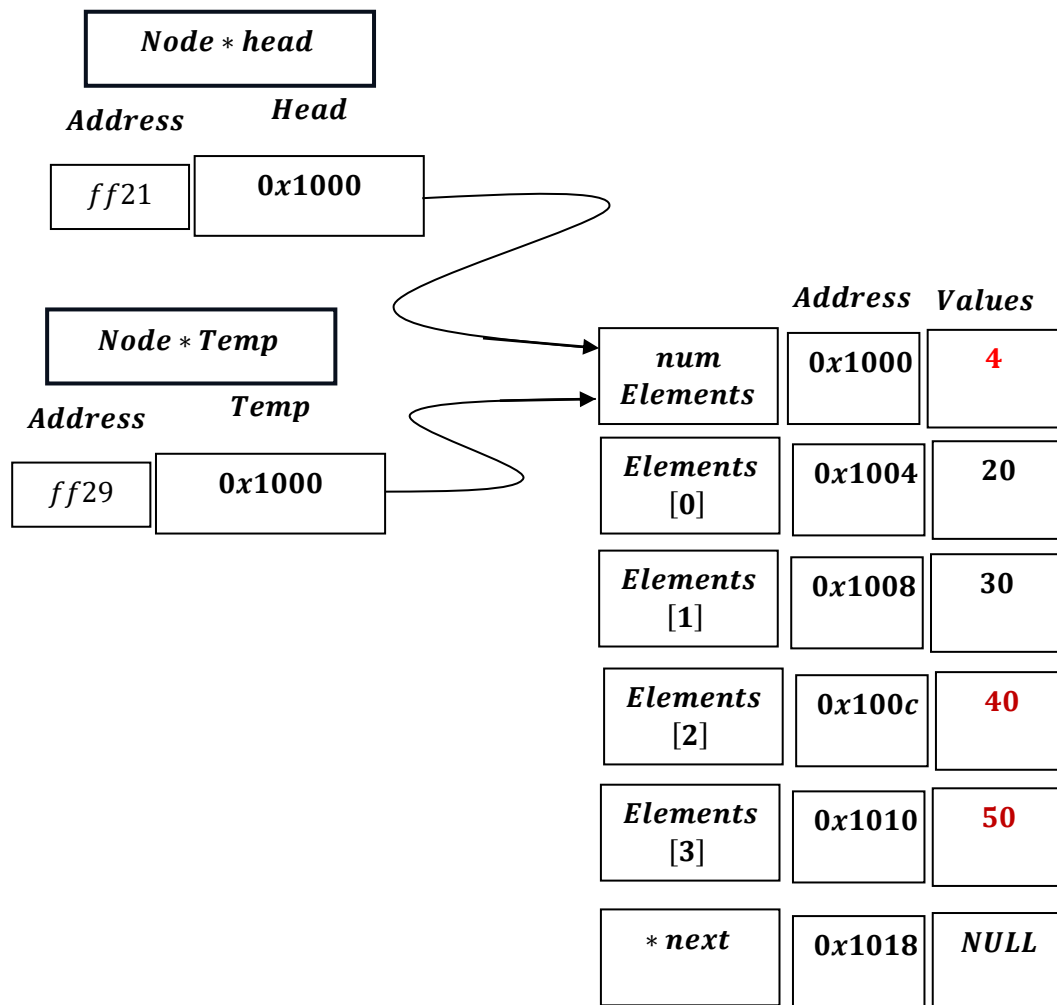
$temp \rightarrow numElements = temp \rightarrow numElements + 1 = 1 + 1 = 2;$

Print: "Inserted " val: 30 " into the existing last node."

[Exit if structure]



Next if we run two(2) times *Insert Element* functions we get:



On next function call:

```
cout << "Enter integer value to insert: ";
cin >> value;
insertElement(value);
break;
```

``Enter integer value to insert:``

say, 60

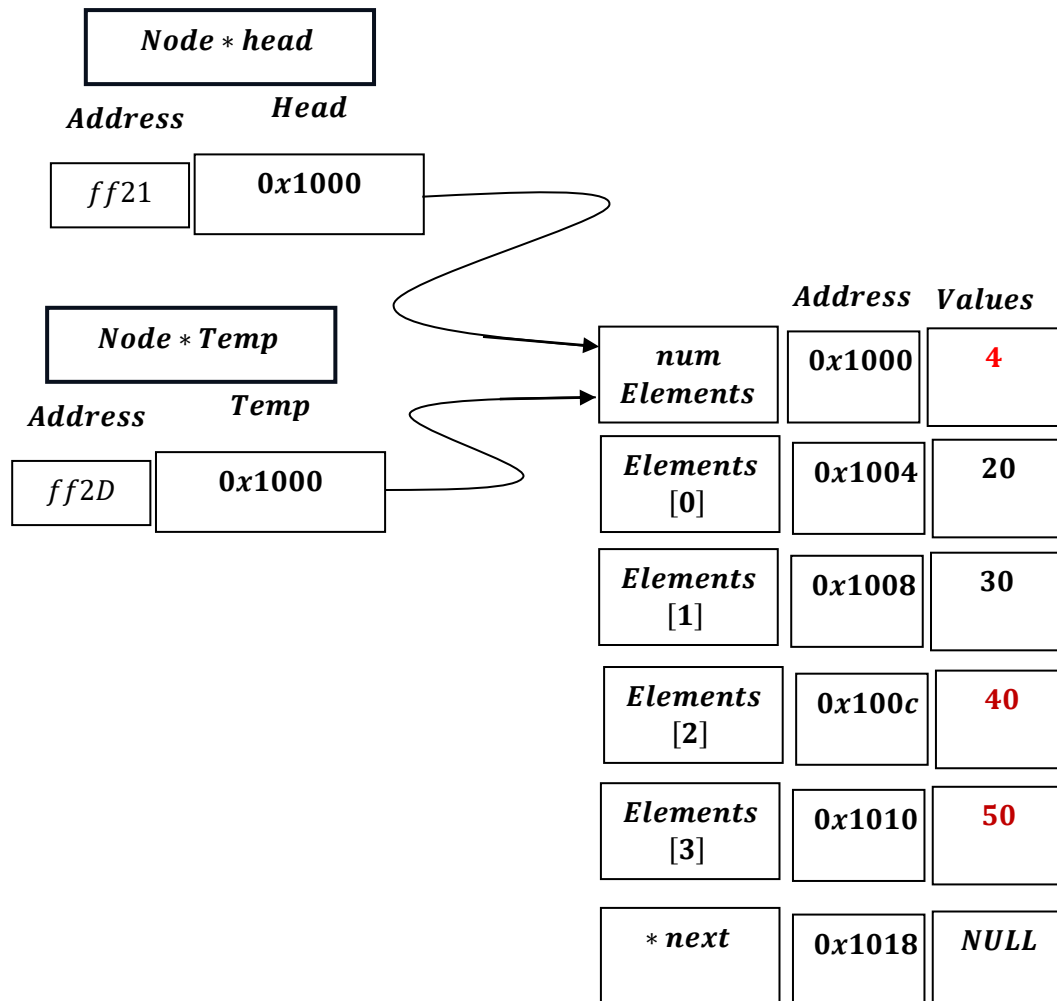
Function Call: *insertElement*(**value**: 60)

```
if (head == NULL)
```

Head $\neq$ NULL [Condition is False]

```
struct Node* temp = head;
```

Temp will be again point to Head



```
while (temp->next != NULL) {  
    temp = temp->next;  
}
```

while (temp: 0x1000 $\rightarrow$ next: 0x1018) $\neq$ NULL is False , hence condition is false $\therefore$ [Loop Exits].

```
if (temp->numElements < MAX_ELEMENTS)
```

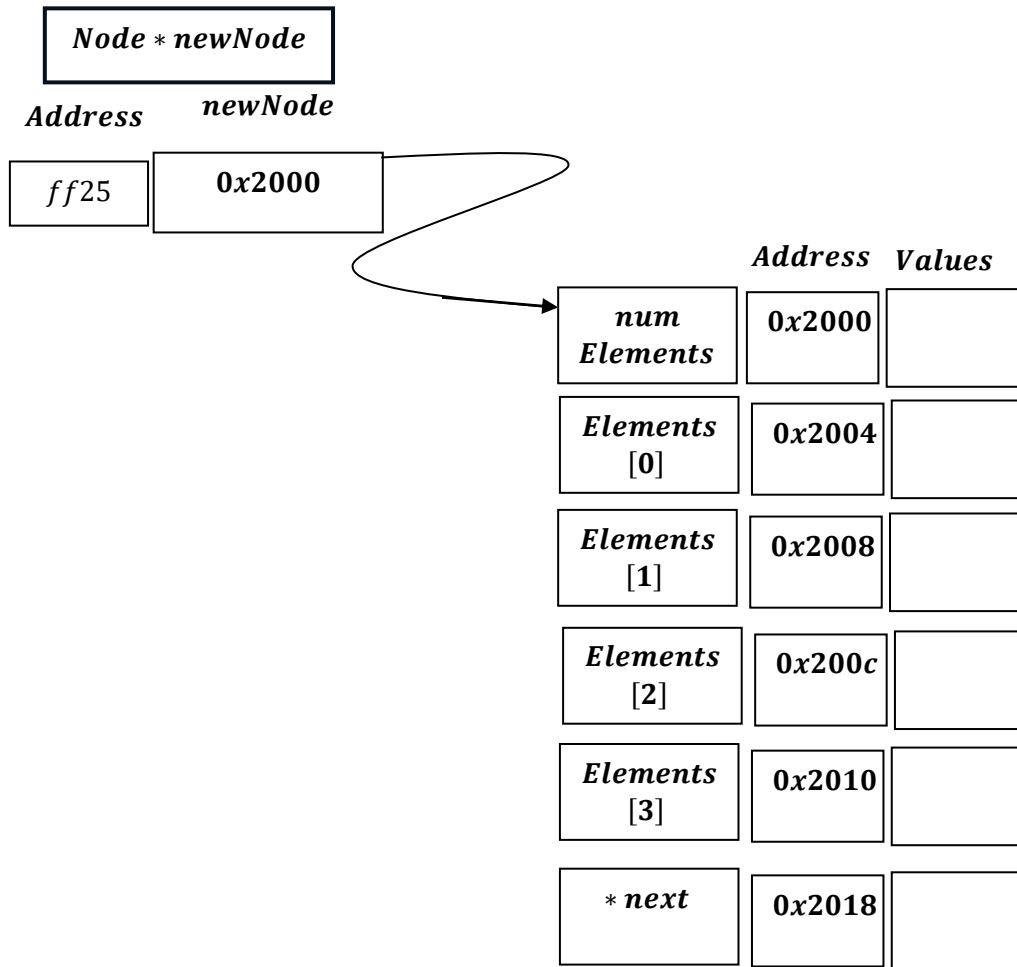
temp → *numElements*: 4 < *MAX_ELEMENTS* : 4 $\left[\begin{array}{l} \text{if condition false,} \\ \text{hence goes to} \\ \text{Else part.} \end{array} \right]$

```
else {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->numElements = 1;
    newNode->elements[0] = val;
    newNode->next = NULL;

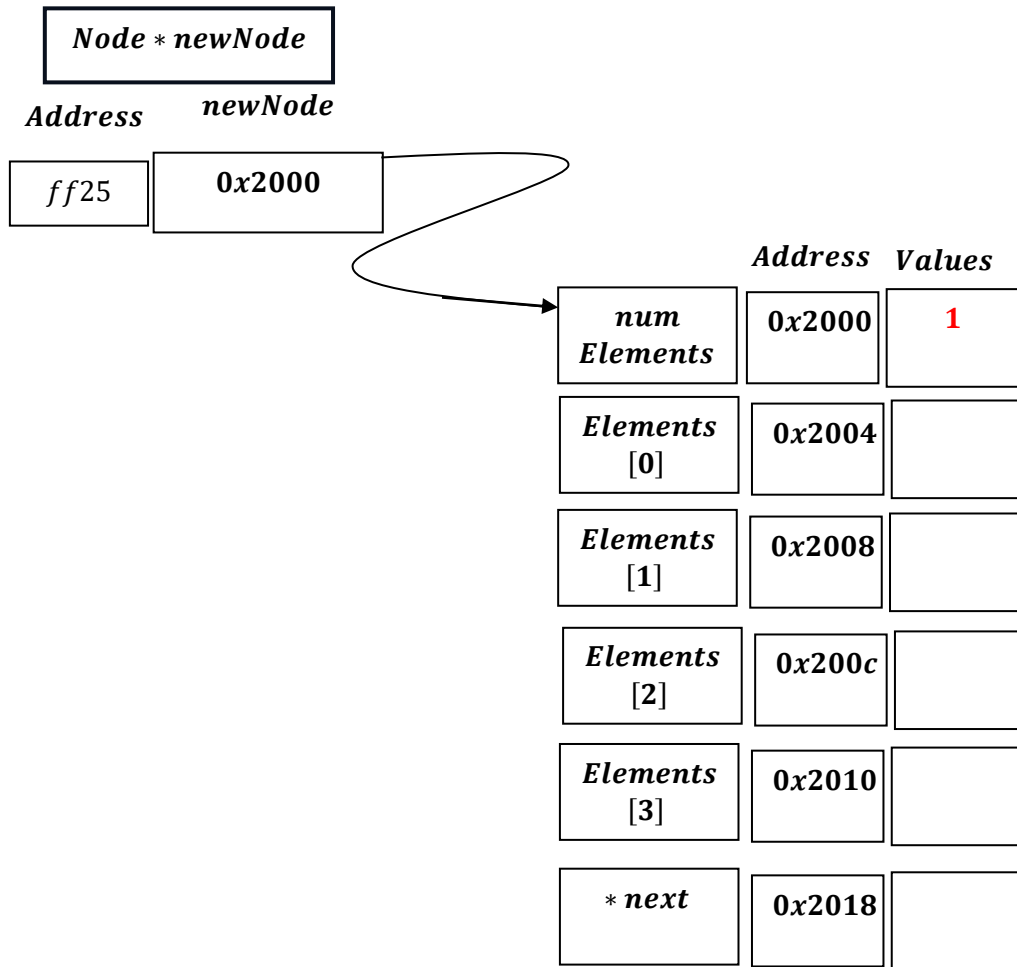
    temp->next = newNode;
    cout << "Node full. Inserted " << val << " into a newly
created node.\n";
}
```

i.e.

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
```



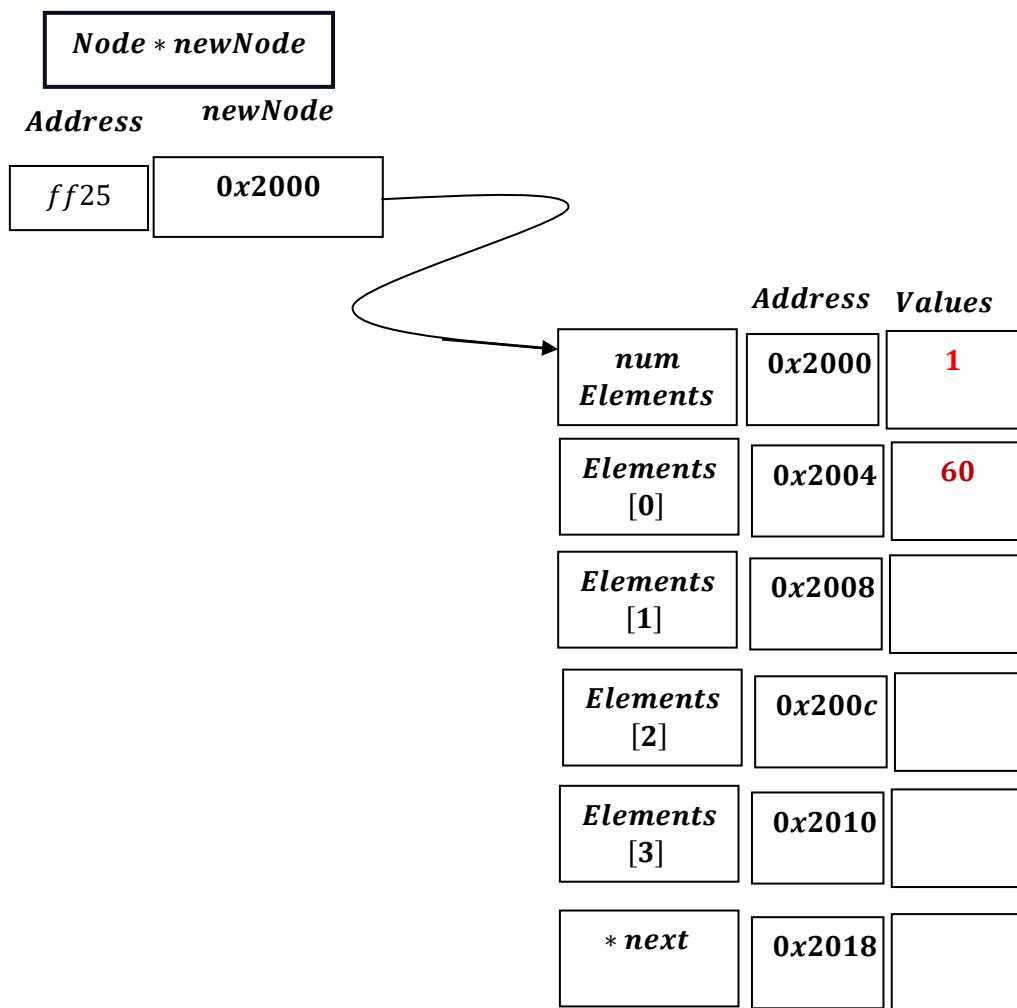
```
newNode->numElements = 1;  
newNode → numElements = 1.
```



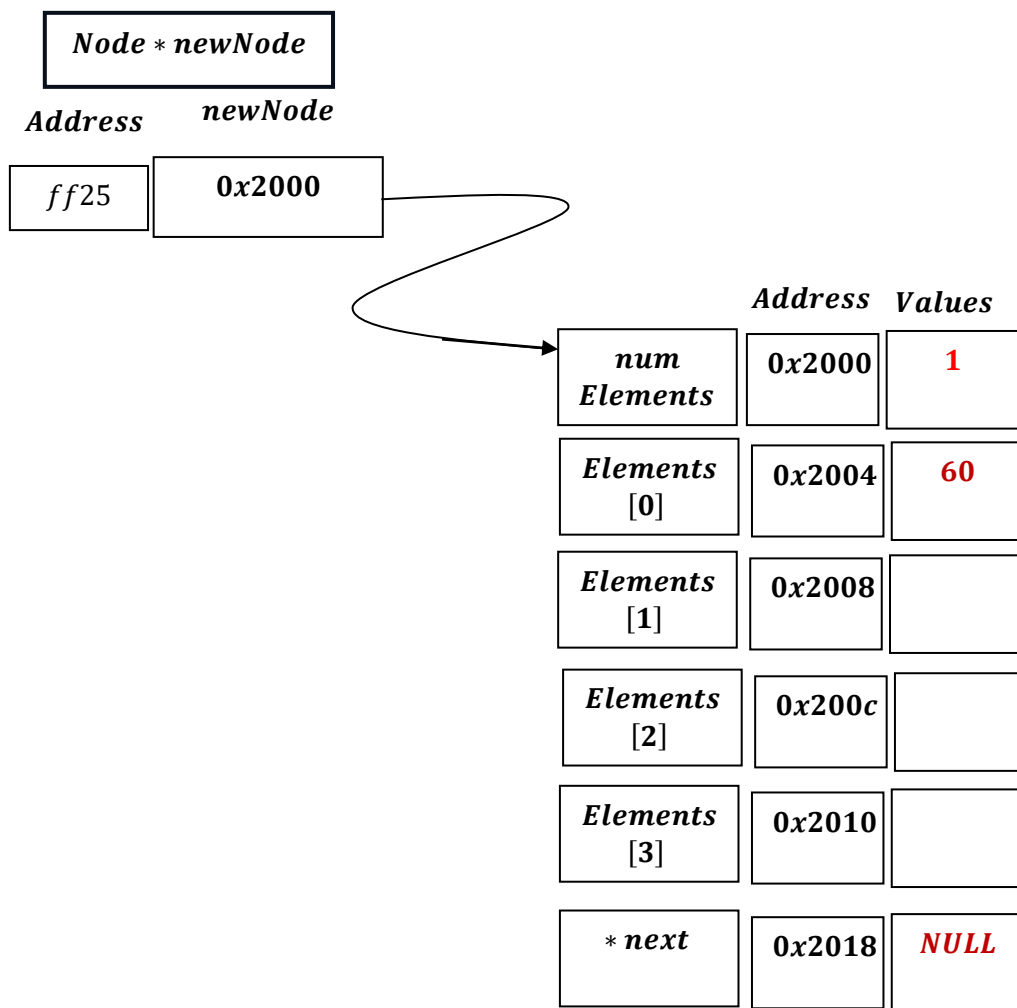
```
newNode->elements[0] = val;
```

Function Call: insertElement(*value*: 60)

newNode → *elements*[0] = *val* = 60.

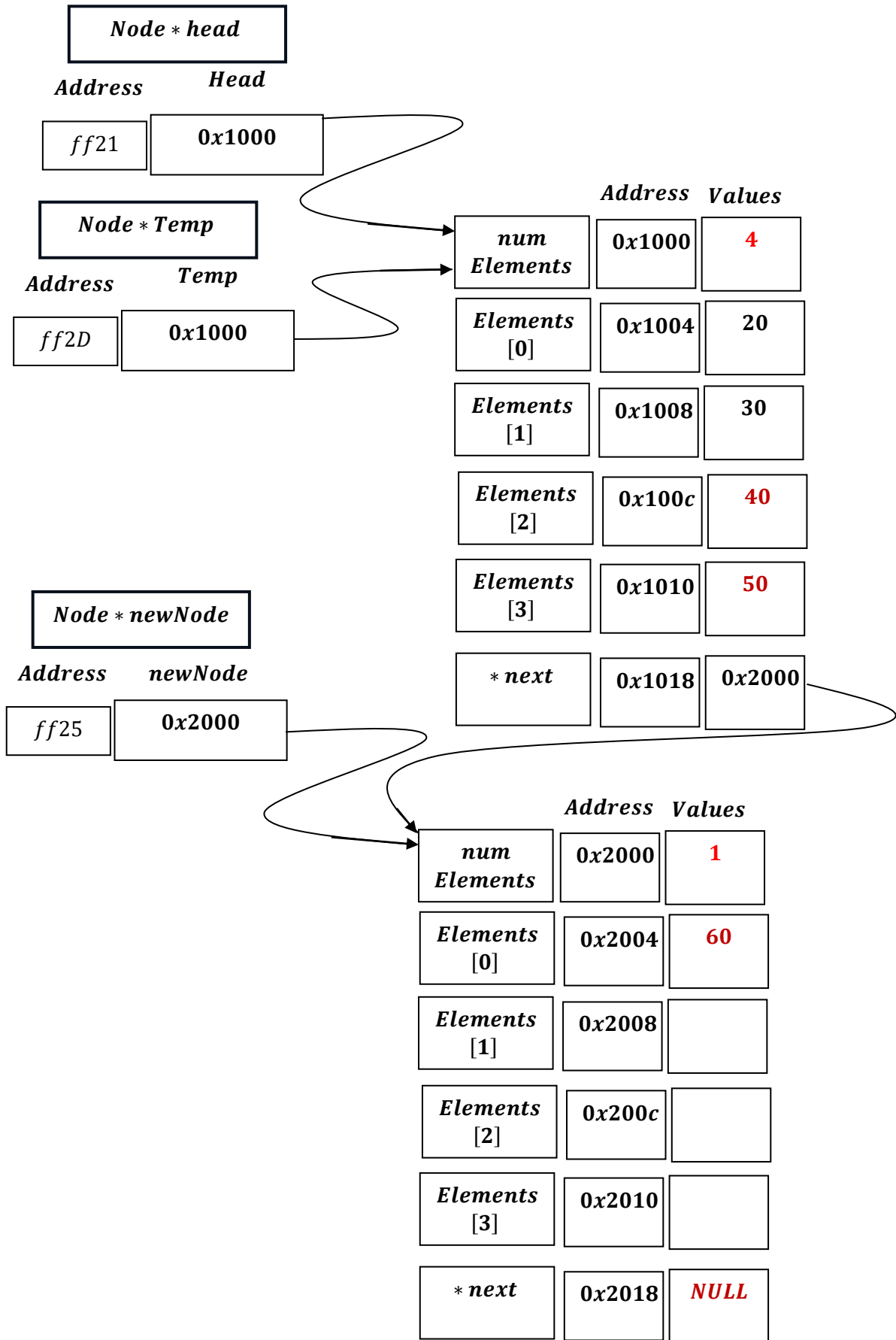


```
newNode->next = NULL;
```



```
temp->next = newNode;
cout << "Node full. Inserted " << val << " into a newly
created node.\n";
```

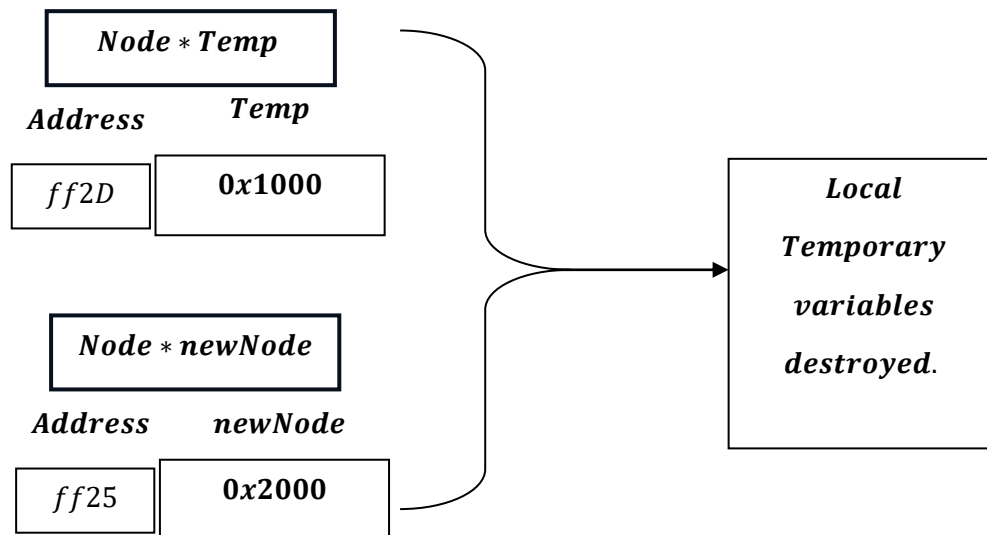
Temp → next = newNode = 0x2000.



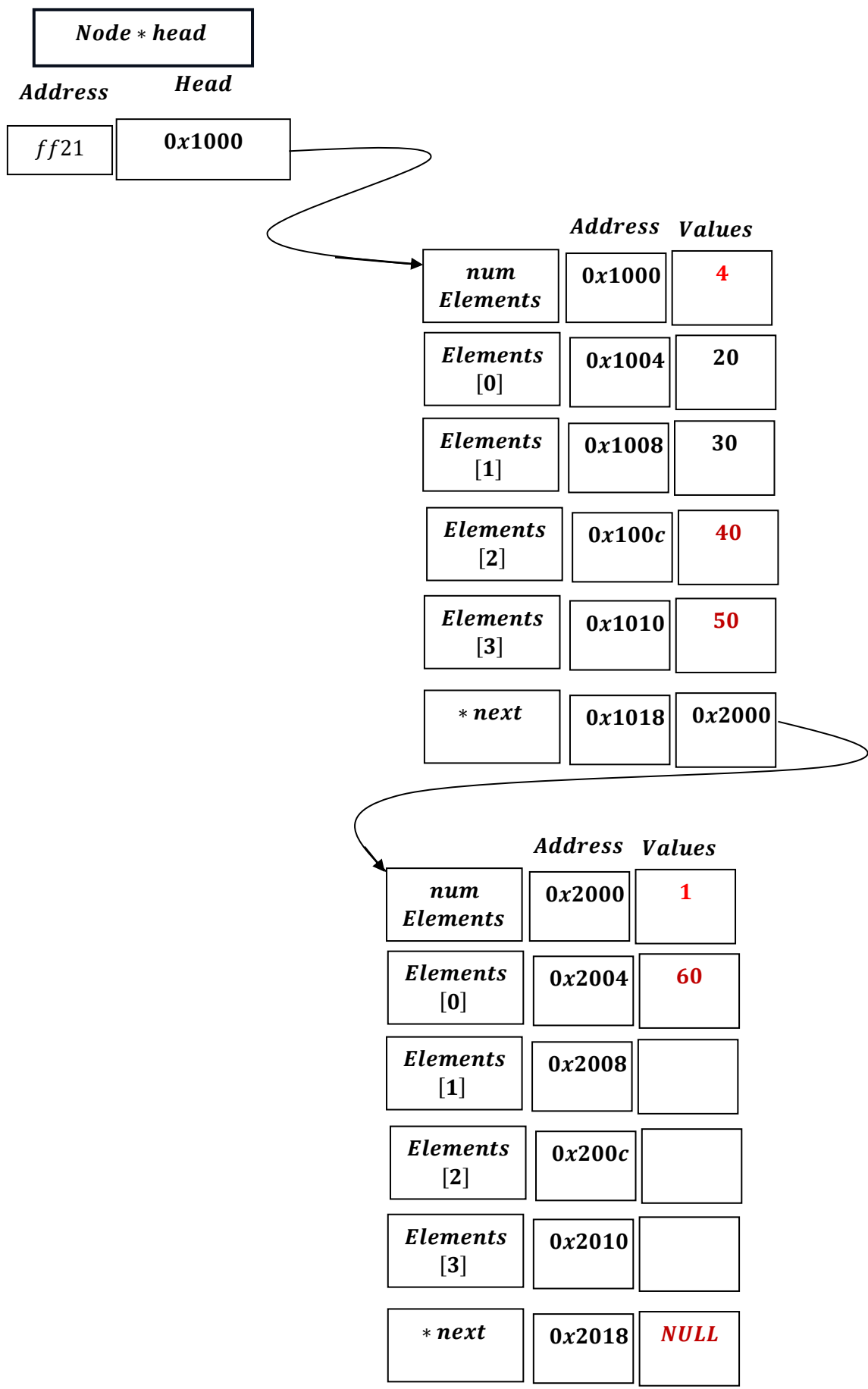
```
cout << "Node full. Inserted " << val << " into a newly  
created node.\n";
```

Print : `` Node full.Inserted `` val: 60 ``into a newly created node.``

As function ends, temporary local variables ends.



And we are left with:



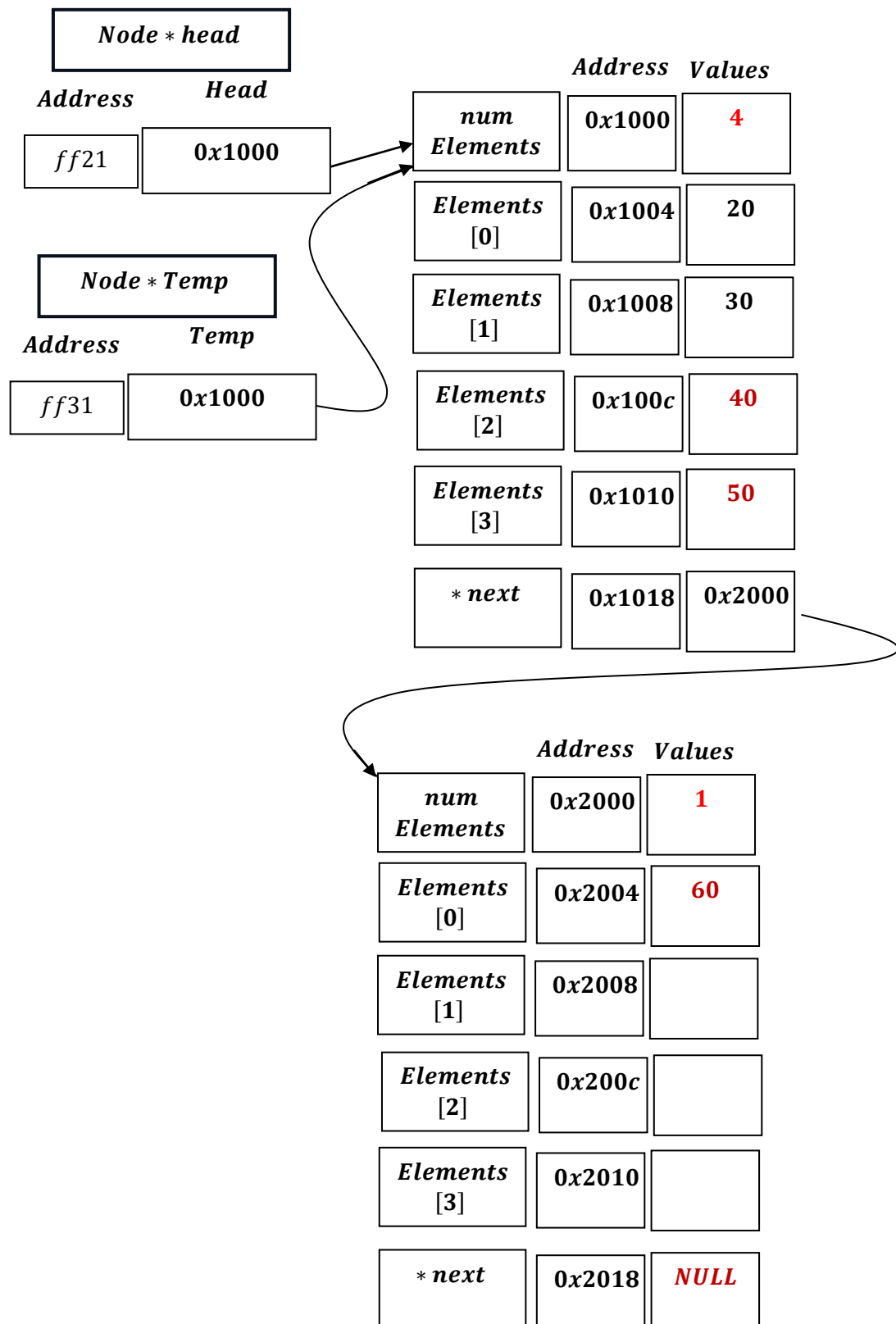
Next if we try to enter another value: **70**.

```
if (head == NULL)
```

Head $\neq$ *NULL* [Condition is False]

```
struct Node* temp = head;
```

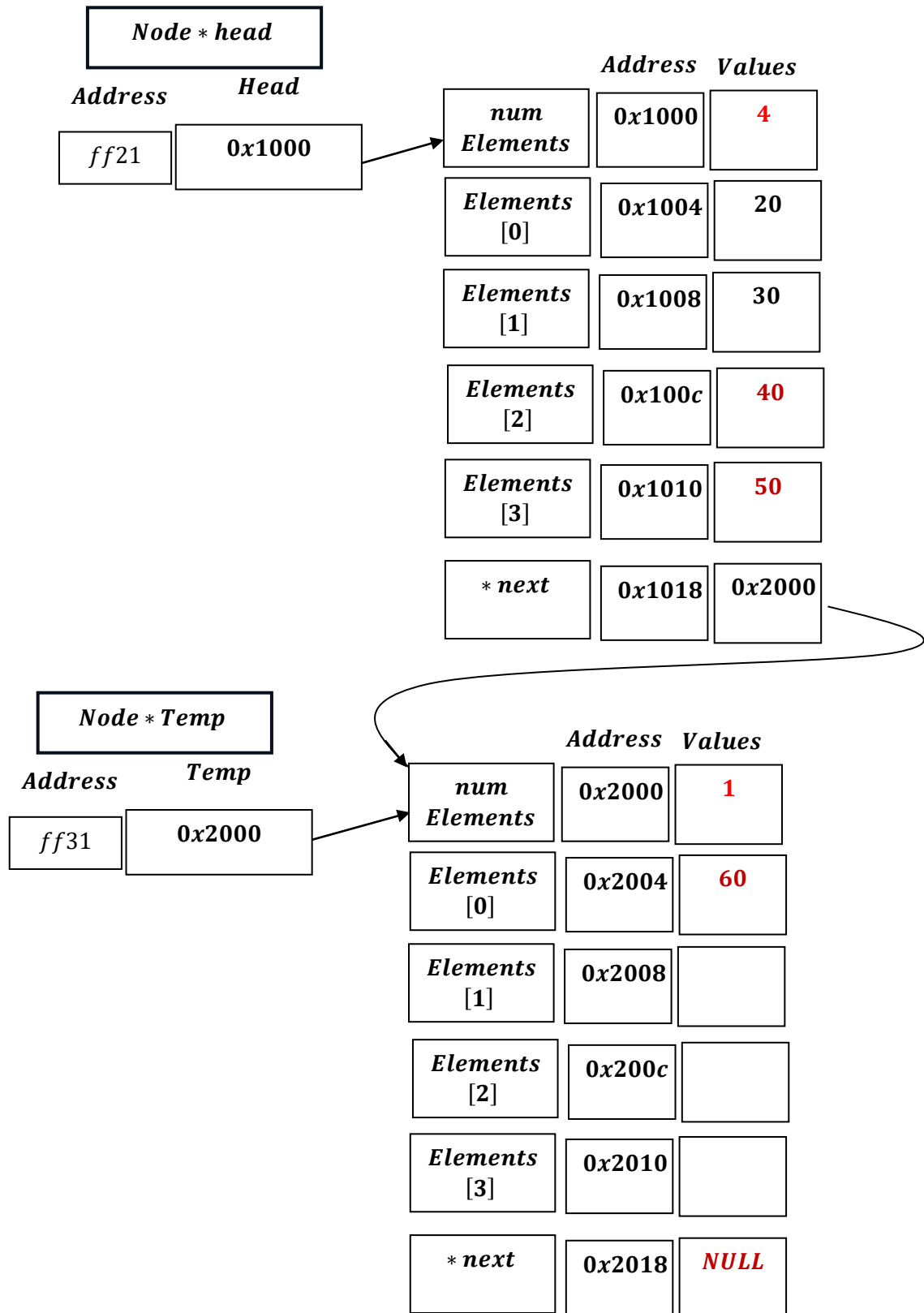
Temp will be again point to *Head*



```
while (temp->next != NULL) {  
    temp = temp->next;  
}
```

While(Temp: 0x1000 → Next : 0x2000 ≠ NULL) $\left[\begin{array}{c} \text{Condition is} \\ \text{True} \end{array} \right] :$
TEMP = TEMP → NEXT : 0x2000

While(Temp: 0x2000 → Next : NULL ≠ NULL) $\left[\begin{array}{c} \text{Condition is} \\ \text{False} \end{array} \right] :$



```

    if (temp->numElements < MAX_ELEMENTS) {
        temp->elements[temp->numElements] = val;
        temp->numElements++;
        cout << "Inserted " << val << " into the existing last
node.\n";
    }

```

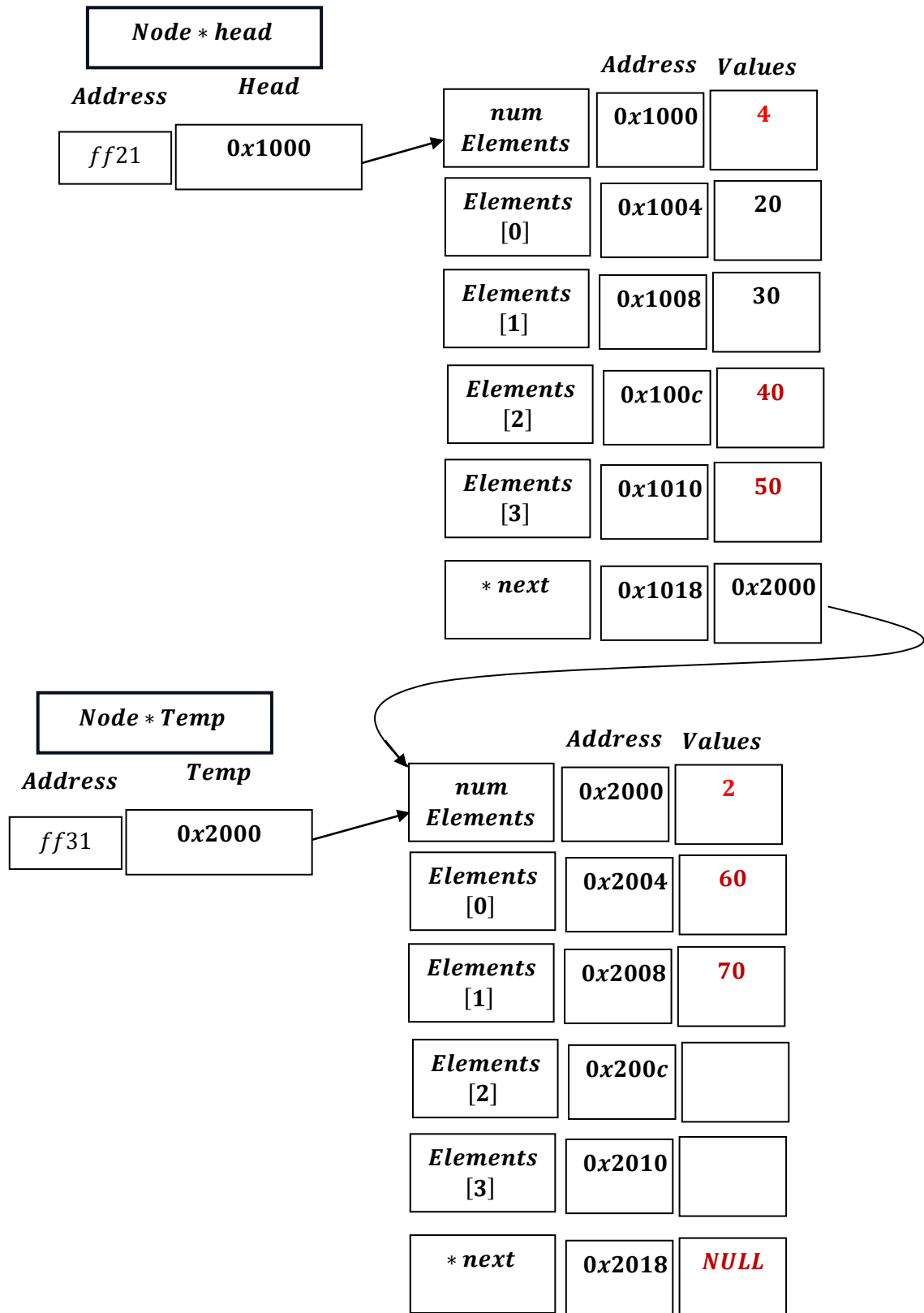
if (*temp*: 0x2000 $\rightarrow$ *numElements* = 1 < *MAX_ELEMENTS* = 4) $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{True} \end{array} \right]$

temp $\rightarrow$ *elements*[*temp* $\rightarrow$ *numElements* = 1] = *val* := 70;

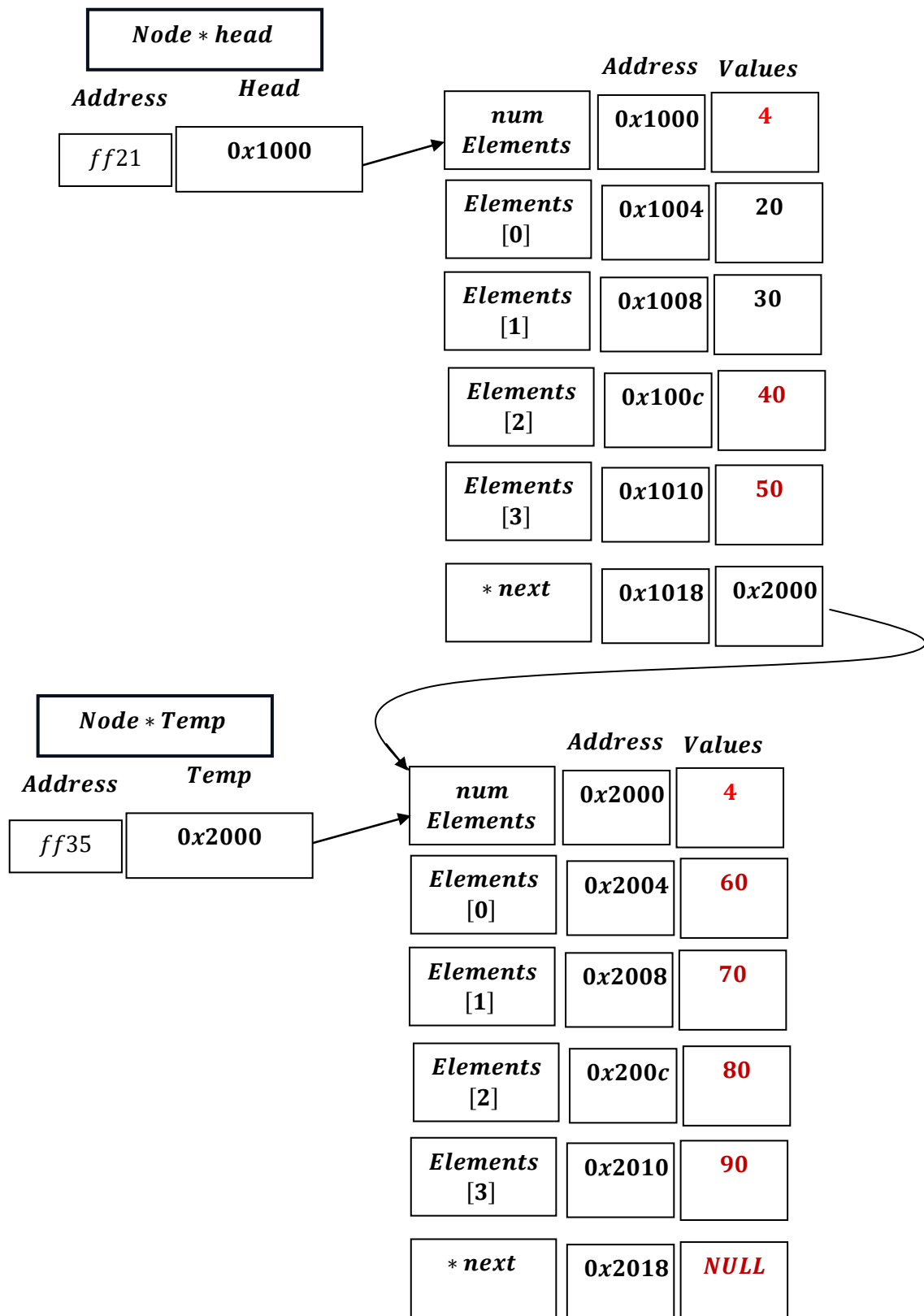
temp $\rightarrow$ *numElements* = *temp* $\rightarrow$ *numElements* + 1 = 1 + 1 = 2;

Print: ``Inserted `` *val*: 70 `` into the existing last node.``

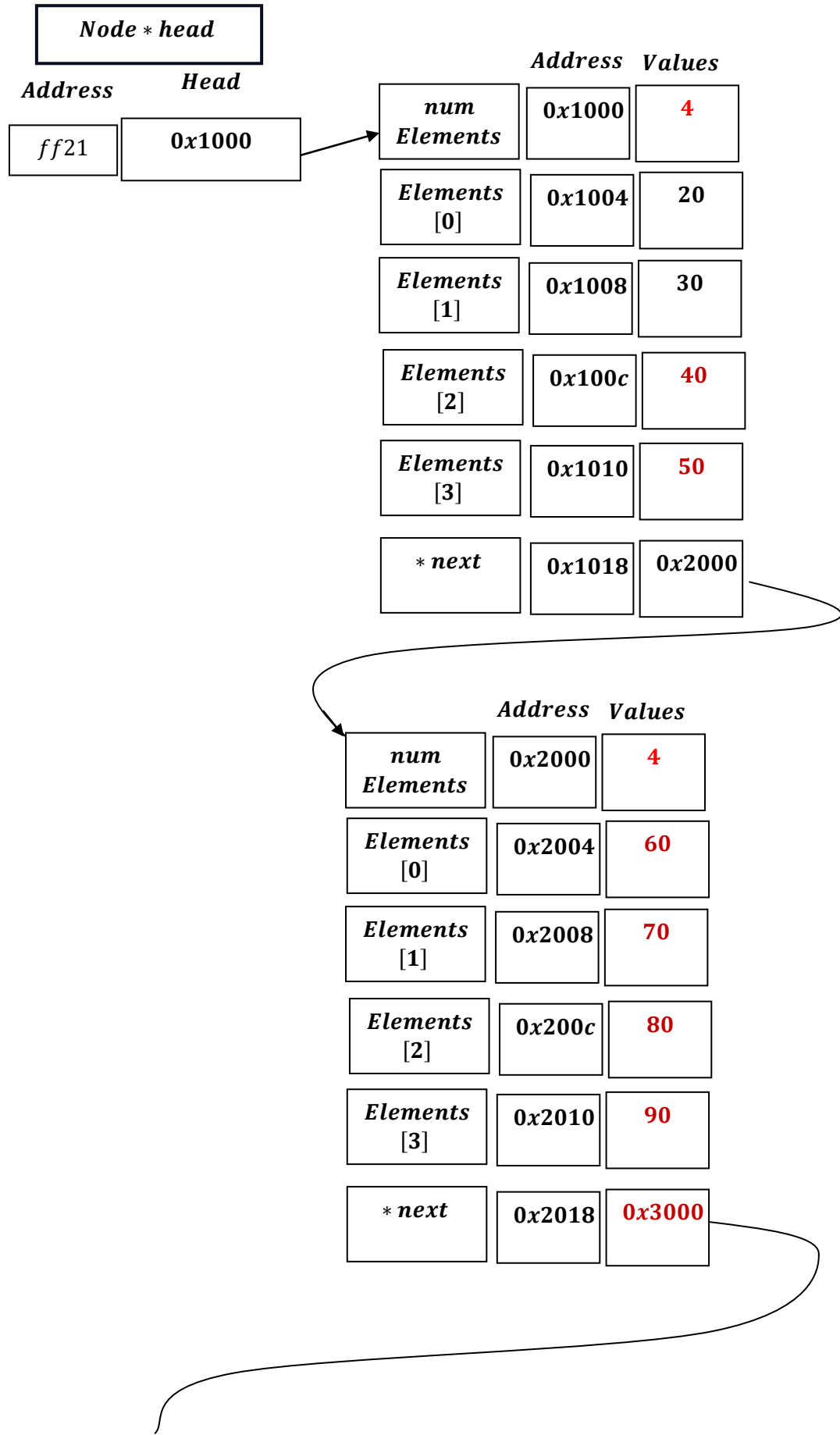
[Exit if structure]

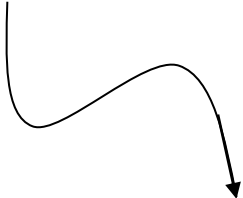


Like this we can get every thing updated as :



As soon as numElements becomes 4 , newNode is created and inserted:





	<i>Address</i>	<i>Values</i>
<i>num</i> <i>Elements</i>	0x3000	4
<i>Elements</i> [0]	0x3004	100
<i>Elements</i> [1]	0x3008	110
<i>Elements</i> [2]	0x300C	120
<i>Elements</i> [3]	0x3010	130
* <i>next</i>	0x3018	NULL
